

Scripting in a shell

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM05.html



Quick links:

Click on any [🏠](#) to get back here!

- the [command line interface](#)
- what is a [shell](#)?
- [which shells](#) do I have?
- keyboard [shortcuts](#)
- [how to use a shell](#)
- [types](#)
- [a login or an interactive shell](#)?
- what's in `.bashrc`
- making an [alias shortcut](#)
- [a simple awk script](#)
- [add and commit](#) my script using git
- [system variables](#)
- [IFS](#): the internal field separator
- using [escape characters](#)
- [simple maths](#)
- a simple `while read` loop
- a more complicated `while read` loop
- `for` loops: simple or nested
- [conditionals](#): if
- [git](#) time again!
- [plain text drawing](#) of git repository timelines
- [git adog](#)
- [git adog2](#) adog with more colour

Exercises

1. [Initialise, configure, add/commit, and push](#) a new repo to GitHub

Give feedback
Ask for help

- [default variable values](#)
- [string manipulation](#)
- [bash special characters](#)
- [importing variables](#) into `awk`

2. [That exercise again](#): process `example_people_data.tsv` file using `bash`, not `awk`

🏠 The Command Line Interface

Today we will learn....

1. more about typing commands into a terminal window
2. how there are several different terminal shells
3. about variables
4. about loops
5. about variables in loops
6. about conditional statements
7. about variables in conditional statements in loops
8. ...

🏠 **What is a shell?**

- what you saw when you started MobaXterm or Terminal: a "terminal" shell
- what you saw when you logged into the MSc server
- provides an interface environment to the Linux "kernel" core of the operating system
- interprets commands from the keyboard or file and tells the kernel what to do

Several kinds of shell

Several kinds of shell exist that all essentially do the same kinds of things, but have slightly different syntax

- BASH (**B**ourne-**A**gain **S**hell) : usual/default shell in most Linux installations, named after Stephen Bourne; many online (help) resources available, e.g. <http://tldp.org/LDP/Bash-Beginners-Guide/html/> and <https://www.freecodecamp.org/news/bash-scripting-tutorial-linux-shell-script-and-command-line-for-beginners/>.
- CSH (C SHell) : syntax and usage are very similar to the C programming language
- TCSH enhanced version of CSH, but with programmable command-line completion, command-line editing, and a few other features
- KSH (Korn SHell) - Created by David Korn at AT & T Bell Labs; the base for the POSIX Shell standard specifications.

What is POSIX?

[Wikipedia:](#)

"The Portable Operating System Interface (POSIX) is a family of standards specified by the Institute of Electrical and Electronics Engineers (IEEE, "I triple E") Computer Society for maintaining compatibility between operating systems. POSIX defines the application programming interface (API), along with command line shells and utility interfaces, for software compatibility with variants of Unix and other operating systems."

📌 What shells are available in my installation?

cat /etc/shells

```
# /etc/shells: valid login shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/bin/dash
/usr/bin/dash
/bin/zsh
/usr/bin/zsh
/usr/bin/tmux
/usr/bin/screen
/usr/bin/sh
/bin/ksh93
/usr/bin/ksh93
/bin/rksh93
/usr/bin/rksh93
/bin/tcsh
/usr/bin/tcsh
```

Which one do I have set up as default?

echo \$SHELL

```
/bin/bash
```

Get used to doing it with curly brackets {}

echo \${SHELL}

```
/bin/bash
```

As always, lots of help online, for example

- [The Bash Reference Manual](#) and
- [bash scripting tutorials](#)
- <https://www.codecademy.com/catalog/language/bash>

Run a simple program

cal

```
September 2025
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30
```

Inline help too (even when you get it wrong)

cal --help

```
cal: invalid option -- '-'
Usage: cal [general options] [-jy] [[month] year]
       cal [general options] [-j] [-m month] [year]
       ncal -C [general options] [-jy] [[month] year]
       ncal -C [general options] [-j] [-m month] [year]
       ncal [general options] [-bhJjpwySM] [-H yyyy-mm-dd] [-s country_code] [[month] year]
       ncal [general options] [-bhJeoSM] [year]
General options: [-31] [-A months] [-B months] [-d yyyy-mm]
```

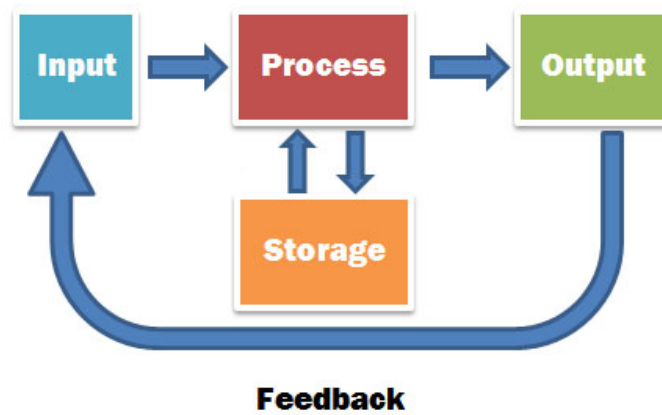

📌 Useful keyboard shortcuts when typing in a shell

- Tab : **Auto-completes** file/directory/command names etc
- ↑ ↓ : Up/down arrow keys to view previous commands (or use the [history](#) command...)
- CTRL+ **A** : Beginning of the line.
- CTRL+ **E** : End of the line.
- CTRL+ **L** : Clear the screen.
- CTRL+ **U** : Clear the line i.e. Delete all words from command line.
- CTRL+ **W** : Delete the word starting at cursor.
- CTRL+ **T** : Swap the last two characters before the cursor.
- CTRL+ **H** : Delete the character to the left of the cursor.
- CTRL+ **R** : Search through previously used commands

- CTRL+ **C** : Cancel currently running command

📌 How can we effectively use the shell to do things?

My favourite picture:



We could type a command, hit return, and then watch the output, then do the next one...

Single actions: may be OK, but what about our strings of actions, for the first optional ICA ([My First Pipeline](#)) you got given, and are now contemplating how to tackle...!?

We could have a collection of action commands in a plain text file, that we then run, "hands-free", as a programme in a shell: a "shell script".

```
# My stylised "pseudocode" way to do cool things
```

```
# version 1
```

```
# do this for a list of datasets
```

```
start in homespace  
inputdata from somewhere_0 to command1 then output to somewhere_1  
inputdata from somewhere_1 to command2 then output to somewhere_2
```

```
# Probably need a check here?
```

```
inputdata from somewhere_2 and homespace to command3  
then output final result to homespace again
```

OR...

we could have individual action commands in several files, that we then run,
"calling" the next script when it is finished

My stylised pseudocode way to do cool things

version 2

do this for a list of datasets

```
start in my homespace  
run shell_script1 (invoked by me/user)  
run shell_script2 (invoked by shell_script1)  
run shell_script3 (invoked by shell_script2)  
output final results to homespace
```

Which might be "best"?

Do you think the user (you/someone else) wants to invoke each one in turn
(usually not, I'd say...!)?

Which one is easier to maintain in our `git` repository?



Which one should you use for My First Pipeline?

📌 Types

The `type` command can be used to find out if a command is a bash "built in" or an external binary file.

More details about the `type` command can be found [here](#).

`type -a echo`

```
echo is a shell builtin
echo is /usr/bin/echo
echo is /bin/echo
```

`type -a date`

```
date is /usr/bin/date
date is /bin/date
```

`type -a ls`

```
ls is aliased to `ls --color=auto'
ls is /usr/bin/ls
ls is /bin/ls
```

`type -a ll`

```
ll is aliased to `ls -alrt'
```

📌 What happens when we log in?

The bash shell can use some/all of the following:

- `/etc/profile` : the systemwide initialization file, executed for login shells
- `/etc/bash.bashrc` : non-standard, system-wide per-interactive-shell startup file (we don't have it on our installation)
- `${HOME}/.bash_profile` : the personal initialization file, executed for login shells
- `${HOME}/.bashrc` : the individual per-interactive-shell startup file executed for non-login shells
- `${HOME}/.inputrc` : individual readline initialization file
- `/etc/bash.logout` : the system-wide login shell cleanup file, executed when a login shell exits
- `${HOME}/.bash_logout` : the individual login shell cleanup file, executed when a login shell exits

The programme/script `/etc/profile` runs first when a user logs in

Have a look at what is in /etc/profile

more /etc/profile

```
# /etc/profile: system-wide .profile file for the Bourne shell (sh(1))
# and Bourne compatible shells (bash(1), ksh(1), ash(1), ...).

if [ "${PS1-}" ]; then
  if [ "${BASH-}" ] && [ "$BASH" != "/bin/sh" ]; then
    # The file bash.bashrc already sets the default PS1.
    # PS1='\h:\w\$ '
    if [ -f /etc/bash.bashrc ]; then
      . /etc/bash.bashrc
    fi
  else
    if [ "`id -u`" -eq 0 ]; then
      PS1='# '
    else
      PS1='$ '
    fi
  fi
fi

if [ -d /etc/profile.d ]; then
  for i in /etc/profile.d/*.sh; do
    if [ -r $i ]; then
      . $i
    fi
  done
  unset i
fi
```

Can we change the contents of /etc/profile ?

ls -al /etc/profile

```
-rw-r--r-- 1 root root 582 Sep 10 2025 /etc/profile
```



Many/most of the important shells we can use are listed in the `/etc/shells` file

Show the contents of `/etc/shells`

`cat /etc/shells`

```
# /etc/shells: valid login shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/bin/dash
/usr/bin/dash
/bin/zsh
/usr/bin/zsh
/usr/bin/tmux
/usr/bin/screen
/usr/bin/sh
/bin/ksh93
/usr/bin/ksh93
/bin/rksh93
/usr/bin/rksh93
/bin/tcsh
/usr/bin/tcsh
```

1. `${HOME}/.bash_profile`
 - which calls `${HOME}/.bashrc`
 - which calls `/etc/bashrc (/etc/bash.bashrc)`
2. `${HOME}/.bash_login`
3. `${HOME}/.profile`

📌 What's in a .bashrc file?

.bashrc is kept in our home directory

cd

pwd

/home/s0000000

Have a quick look, most of you will get this result

cat .bashrc

Nothing!

What DO we have in our home directory (something like this perhaps)?

ls -alrt

```
total 56
drwxr-xr-x  3 s0000000 users 4096 Sep 20 12:24 .cache
drwxr-xr-x  3 s0000000 users 4096 Sep 20 12:24 .config
drwxr-xr-x 225 root      lcfc  4096 Sep 23 16:54 ..
-rw-----  1 s0000000 users  207 Sep 30 10:08 .Xauthority
-rw-r--r--  1 s0000000 users  103 Sep 30 12:34 .gitconfig
drwx-----  5 s0000000 users 4096 Sep 30 12:34 .
drwxr-xr-x  4 s0000000 users 4096 Sep 30 12:42 Exercises
-rw-----  1 s0000000 users 18280 Sep 30 13:10 .bash_history
...
```

Do we have any bash files there, file names that start with .b ?

ls .b*

.bash_history

What might be in it (assuming you HAVE done the all the stuff from last Friday)?

tail -1 .bash_history

git clone https://github.com/B000000-2019/hello-world Als_HelloWorld

or perhaps try ?

grep "World" .bash_history

Tried it again on my own personal server

cat .bashrc

```
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *i*) ;;
    *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

...
```

🏠 Let's make an alias (shortcut)!

We are going to do this next bit to illustrate how the same "programme" can be different things in different places, so it's always good to know where you are!

```
# Make sure we are in our homespace
```

```
cd
```

```
# We'll put some aliases in .bash_profile for now (could use .bash_aliases)
```

```
# If .bash_profile doesn't exist, we can create an empty file using touch
```

```
# If .bash_profile is already there, it gets a fresh datestamp
```

```
touch .bash_profile
```

```
# What .bash files are there now?
```

```
ls -al .bash*
```

```
-rw- - - - - 1 s0000000 users 4037 Sep 30 11:06 .bash_history  
-rw- r--r-- 1 s0000000 users    0 Sep 30 11:06 .bash_profile
```

```
# Several ways to add content to the file...
```

```
# Today we are going to use the gedit editor and see how slow it is over this connection!
```

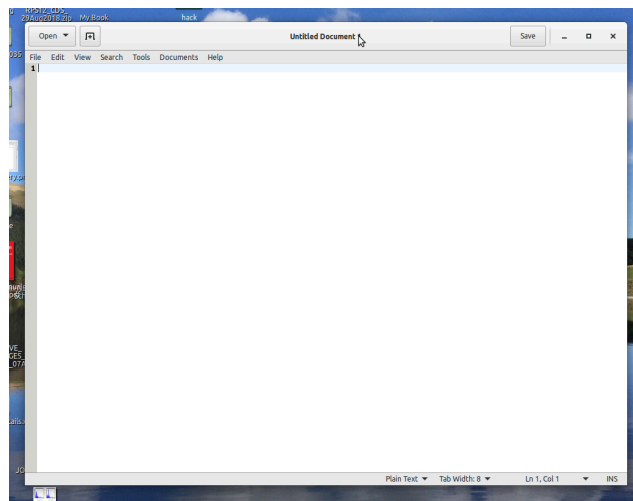
```
# If you are on a Mac and just using terminal
```

```
# THIS WILL NOT WORK FOR YOU!
```

```
gedit .bash_profile &
```

```
[1] 8193
```

The screen, once it finally appears (? 30 seconds ?), will look something like this:



Add the following text (just copy paste from this web page!)

```
#!/bin/bash
# My first comment line
#
# This is just for fun
alias a='echo -e "\n###\n I love Linux and doughnuts\n###"'
# This helps stop me deleting things I shouldn't: interactive yes/no
alias rm='rm -i'
# This also helps stop me "deleting" things: see mv --help
alias mv='mv -i'
```

Save and exit from gedit

Check the contents

cat .bash_profile

```
#!/bin/bash
# My first comment line
#
# This is just for fun
alias a='echo -e "\n###\n I love Linux and doughnuts\n###"'
# This helps stop me deleting things I shouldn't: interactive yes/no
alias rm='rm -i'
# This also helps stop me "deleting" things: see mv --help
alias mv='mv -i'
```

What .bash* files are there now?

ls -al .bash*

```
-rw----- 1 s0000000 users 4037 Sep 30 11:08 .bash_history
-rw-r--r-- 1 s0000000 users 172 Sep 30 11:08 .bash_profile
```

Let's try it out, do we execute the profile?

.bash_profile

```
-bash: .bash_profile: command not found
```

./bash_profile

```
-bash: ./bash_profile: Permission denied
```

```
ls -al .bash_profile
```

```
-rw-r--r-- 1 s0000000 users 172 Sep 30 11:08 .bash_profile
```

```
chmod 700 .bash_profile
```

```
ls -al .bash*
```

```
-rw----- 1 s0000000 users 4151 Sep 30 11:09 .bash_history  
-rwx----- 1 s0000000 users 172 Sep 30 11:08 .bash_profile
```

```
# Let's try to execute (run) .bash_profile again
```

```
./bash_profile
```

```
# Run the command a
```

```
a
```

```
-bash: a: command not found
```

```
# Let's read in ("source") .bash_profile
```

```
source .bash_profile
```

```
# Run the command a which, this time, uses the alias we have just set up
```

```
a
```

```
###  
I love Linux and doughnuts  
###
```

```
# Now let's start an interactive bash shell inside our login shell
```

```
bash
```

```
# Does it use .bash_profile ?
```

```
# Is the "a" alias still there? NO
```

```
a
```

```
-bash: a: command not found
```

```
# Interactive shells use .bashrc, so let's add a line to
```

```
# .bashrc using echo (or you could just use vi or nano)
```

```
# and make a do something else!
```

```
echo "alias a='echo -e \"### Actually, I only love doughnuts ###\"' " >>  
.bashrc
```

Run the command `a`

`a`

```
### Actually, I only love doughnuts ###
```

Exit the interactive bash shell, go back into the login shell

`exit`

`exit`

Is the new alias still there in the login shell? No, but the initial one is...

`a`

```
###  
  I love Linux and doughnuts  
###
```

📌 What was the `#!/bin/bash` thing all about?

It is called the "bang" line or "shebang" line.

The `#!` syntax is used in scripts to indicate an interpreter for execution under Linux operating systems.

Most Linux shell, perl, python scripts (should) start with it: tells the kernel what interpreter to use!

```
#!/bin/bash
```

or

```
#!/usr/bin/perl
```

or

```
#!/usr/bin/python3
```

How do we find out where the programme being used is running from?

which awk

/usr/bin/awk

What has "happened" when we ran this example command?
--

awk '(\$4 > 100)' someinputfile

The Sun is 150 million kilometres away.
--

📌 Let's put together a simple awk script!

```
# Did we set up a git repository in our Exercises folder? YES
```

```
ls -al ${HOME}/Exercises/
```

```
total 32
drwxr-xr-x 1 aivens2 864524   196 Sep 26 17:45 .
drwx----- 1 aivens2 users 16384 Sep 30 14:34 ..
drwxr-xr-x 1 aivens2 864524   456 Sep 26 21:12 .git
-rw-r--r-- 1 aivens2 864524    54 Sep 26 14:33 .gitignore
drwxr-xr-x 1 aivens2 864524   832 Sep 26 20:00 Lecture04
```

```
# Make a directory for today's work and go to it
```

```
mkdir -p ~/Exercises/Lecture05 ; cd ~/Exercises/Lecture05
```

```
pwd
```

```
/home/s0000000/Exercises/Lecture05
```

Let's write an `awk` script/programme. Start your favourite plain text editor on the server to create and save a new file (e.g. `myawkscript.awk`), then add the following lines.

```
# Every "self executing" awk script would start with:
```

```
#!/usr/bin/awk -f
```

```
# Initial values are specified in the BEGIN block
```

```
# between the BEGIN{} brackets, for example:
```

```
BEGIN {
FS="\t";OFS="_";
}
```

```
# Actions for each line of input specified
```

```
# between the {} brackets
```



```
{  
count++;
```

We wanted to check if the first field from the file isn't a hash symbol

```
if($1 != "#")  
{  
print "Currently doing "count ;
```

**# getting the third and twelfth field from the file
and doing a (somewhat meaningless) calculation**

```
total=total + ($12 * $3)  
}  
}
```

**# Commands only done at the END of ALL inputs
get put between END{} brackets**

```
END {  
print "The total for "count" lines was " int(total) > "awkoutputfile.txt";  
print "Script run complete." >> "awkoutputfile.txt" ;  
print "Script run complete."  
system("ls -alrt *awk*")  
}
```

Assuming we HAD created and saved the above awk script as `myawkscript.awk`, we need an input file for it to work on! Luckily, we have obtained the file called `blastoutput2.out` from `/localdisk/data/BPSM/Lecture05`

Now we'll run the script, and use the file `blastoutput2.out` file as input

Change the permission to make the script

- read/write/executable by the user (4+2+1)
- read/executable by group (4+0+1)
- read/executable by other/world (4+0+1)

```
chmod 755 myawkscript.awk
```

Run the awk script, specifying what path and file it is to process

```
./myawkscript.awk blastoutput2.out
```

OR, if we hadnt copied the file initially, this is good too:

```
./myawkscript.awk /localdisk/data/BPSM/Lecture05/blastoutput2.out
```

The script generates some output to screen

```
Currently doing 1
Currently doing 2
Currently doing 3
...
Currently doing 410
Currently doing 411
Script run complete.
-rwxr-xr-x 1 s0000000 users 731 Sep 30 11:21 myawkscript.awk
-rw-r--r-- 1 s0000000 users 57 Sep 30 11:22 awkoutputfile.txt
```

The script generated some output and puts it in a file

cat awkoutputfile.txt

```
The total for 411 lines was 4738135
Script run complete.
```



Add and commit my script to my git repository

git add myawkscript.awk

git commit -m "My first awk script, Lecture05"

```
git commit -m "My first awk script, Lecture05"
[master bbf825c] My first awk script, Lecture05
1 file changed, 34 insertions(+)
create mode 100755 Lecture05/myawkscript.awk
```

git st

```
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    ../Lecture04/CodeFiles.tar.gz
```

```
Untracked files:
  (use "git add ..." to include in what will be committed)
    ../Lecture04/Als_ICA/
    ../Lecture04/all_the_things_I_did
    ../Lecture04/outs/
    awkoutputfile.txt
    blastoutput2.out
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

git log

```
bbf825c (HEAD -> master) My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeaa I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```


📌 Variables of the system kind

We can use the command `man set` to see info about the system
variables currently maintained by the system

We can use the command `set` in a bash shell to see our info

Start a non-login shell

bash

set | head -35

```
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:expand_aliases:extglob:extquote:force_ign
BASH_ALIASES=()
BASH_ARGC=( [0]="0" )
BASH_ARGV=()
BASH_CMDS=()
BASH_COMPLETION_VERSIONINFO=( [0]="2" [1]="11" )
BASH_LINENO=()
BASH_LOADABLES_PATH=/usr/local/lib/bash:/usr/lib/bash:/opt/local/lib/bash:/usr/pkg/lib/bash
BASH_REMATCH=()
BASH_SOURCE=()
BASH_VERSIONINFO=( [0]="5" [1]="2" [2]="21" [3]="1" [4]="release" [5]="x86_64-pc-linux-gnu" )
BASH_VERSION='5.2.21(1)-release'
COLUMNS=161
COMP_WORDBREAKS=$' \t\n\'><=;|&{: '
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/864524/bus
DEFAULT_XDG_CONFIG_DIRS=/etc/xdg
DEFAULT_XDG_DATA_DIRS=/usr/local/share/:/usr/share/
DIRSTACK=()
DISPLAY=localhost:10.0
EUID=864524
GROUPS=()
GTK_MODULES=appmenu-gtk-module
HISTFILE=/home/aivens2/.bash_history
HISTFILESIZE=500
HISTSZ=500
HOME=/home/aivens2
HOSTNAME=bioinfmsc8.bio.ed.ac.uk
HOSTTYPE=x86_64
IFS=$' \t\n'
LANG=en_GB.UTF-8
LINES=46
LOGNAME=aivens2
```

```
MACHTYPE=x86_64-pc-linux-gnu  
MAILCHECK=60
```

```
# We can view the values of the variables, using the $ sign  
# This is what I will get if I am logged into the msc8 server
```

```
echo ${HOSTNAME}
```

```
bioinfmsc8.bio.ed.ac.uk
```

```
echo ${HOSTTYPE}
```

```
x86_64
```

```
echo ${USER}
```

```
aivens2
```

```
echo ${HOME}
```

```
/home/aivens2
```

The last two will be different for you!!

🏠 Variables can have defaults

These things are called variables: they are named things that can hold different values.

When we write them in our code, we can use `$somevariablename` or `${somevariablename}`, the `${}` version is what you must get into the habit of using...

```
# These just give a blank line because they have no value (and don't exist yet either)
```

```
echo $myfirstvariable
```

```
echo ${myfirstvariable}
```

```
# We can set a default value
```

```
echo ${myfirstvariable:= "BPSM"}
```

```
BPSM
```

```
# We can combine bits together
```

```
echo ${myfirstvariable}
```

```
BPSM
```

```
echo $myfirstvariable" is cool"
```

```
BPSM is cool
```

```
echo ${myfirstvariable}" is cool"
```

```
BPSM is cool
```

```
# We can use one variable to create another
```

```
# Note the use of the "ordinary" () brackets: this bit is done first
```

```
mysecondvariable=$(echo $myfirstvariable" is cool")
```

```
echo mysecondvariable
```

```
mysecondvariable
```

```
echo $mysecondvariable
```

```
BPSM is cool
```

```
echo ${mysecondvariable}
```

```
BPSM is cool
```

```
# Be careful about where the brackets get put!
```

```
echo {$mysecondvariable}
```

```
{BPSM is cool}
```

```
# We can set a new value to myfirstvariable
```

```
echo ${myfirstvariable}
```

```
BPSM
```

```
myfirstvariable="Murchison"
```

```
echo ${myfirstvariable}
```

```
Murchison
```

```
echo ${myfirstvariable:="BPSM"}
```

```
Murchison
```

```
# Clearing variables
```

```
echo ${myfirstvariable:="BPSM"}
```

```
Murchison
```

```
# Use unset WITHOUT the $
```

```
unset myfirstvariable
```

```
echo ${myfirstvariable}
```

```
echo ${myfirstvariable:="BPSM"}
```

```
BPSM
```

```
echo ${myfirstvariable}
```

```
BPSM
```

```
echo $myfirstvariable
```

BPSM

```
unset myfirstvariable
```

```
echo $myfirstvariable
```

```
echo myfirstvariable
```

```
myfirstvariable
```

Question: What happens to the variable and its value when you exit the shell?
Consider the use of the command `export`.

```
# Create and export a variable from the current shell
```

```
export foo="Hello friends!"
```

```
# Create but don't export a variable from the current shell
```

```
bar="See ya later..."
```

```
# What can we see?
```

```
echo ${foo}
```

```
Hello friends!
```

```
echo ${bar}
```

```
See ya later...
```

```
# Start a new shell inside the current shell
```

```
bash
```

```
echo ${foo}
```

```
Hello friends!
```

```
echo ${bar}
```

```
# Exit the current shell, and "go back" to the parent shell
```

```
exit
```

```
exit
```

```
# Now try again, we have ${bar} again
```

```
echo ${bar}
```

```
See ya later...
```


This is essentially the same situation as the " I love Linux and doughnuts " and " Actually, I only love doughnuts " example we did earlier.

📌 Manipulating bits of variables

Sometimes, we might want to get access to parts of the value of a variable, for example, perhaps the first few characters of a text string.

There are many good sources of the information as to how to do this sort of thing, for example <https://www.thegeekstuff.com/2010/07/bash-string-manipulation/>.

```
# Show the value of variable where_am_i
```

```
# or it's default value of Edenburghe
```

```
echo ${where_am_i:=Edenburghe}
```

```
Edenburghe
```

```
echo $where_am_i
```

```
Edenburghe
```

```
echo ${where_am_i}
```

```
Edenburghe
```

```
# Oops, that's not right!
```

```
# Replace only first match
```

```
# ${string/pattern/replacement}
```

```
echo ${where_am_i/e/i}
```

```
Edinburghe
```

```
# This is an example of why you need the {} brackets when using variables!
```

```
echo $where_am_i/e/i
```

```
Edenburghe/e/i
```

```
# Replace all matches using //
```

```
echo ${where_am_i//e/i}
```

```
Edinburghi
```

Has the variable value actually changed? NO, as we only used `echo`

```
echo ${where_am_i}
```

Edenburghe

How "long" is the value in my variable?

```
echo ${#where_am_i}
```

10

We want to keep the new value, and we want it to have the same variable name

```
where_am_i=$(echo ${where_am_i}/e/i)
```

```
echo ${where_am_i}
```

Edinburghe

Get the first few characters

```
# ${string:position:length}
```

```
echo ${where_am_i:5}
```

urghe

```
echo ${where_am_i:5:4}
```

urgh

Having defaults does not reset things that exist already!

```
echo ${where_am_i:=Edenburghe}
```

Edinburghe

```
echo ${where_am_i}
```

Edinburghe

Can I clear the contents (value) of a variable?

```
unset ${where_am_i}
```

```
echo ${where_am_i}
```

Edinburghe

No `$` should be used with the `unset` command

```
unset where_am_i
```

```
echo ${where_am_i}
```



bash also has a few "special characters" that you may well find useful to know about, even if you seldom use them!

These include `$*`, `$@`, `$#`, `$$`, `$!`, `$?`, `$-`, `$_` ; a good set of use examples are detailed in <https://www.thegeekstuff.com/2010/05/bash-shell-special-parameters/>.

I'd really recommend that you spend some time looking at these once you have a reasonable understanding of how bash can be used, they can have many uses that might make your life a WHOLE lot easier when it comes to writing code!

Lots of echoes!

As you have seen, we can use `echo` to output the value of a variable or to print out some text.

```
# An echo line or two
```

```
echo "This is a bit of text"
```

```
This is a bit of text
```

```
echo "3\n4\n5" > myinputfile.tsv
```

```
cat myinputfile.tsv
```

```
3\n4\n5
```

```
echo -e "3\n4\n5" > myinputfile.tsv
```

```
cat myinputfile.tsv
```

```
3  
4  
5
```

```
echo "The date is $(date)"
```

```
The date is Tues 30 Sep 11:44:15 BST 2025
```

```
# Define a few variables
```

```
now=$(date +"%d_%m_%Y")
```

```
echo "Today is ${now}"
```

```
Today is 30_09_2025
```

```
echo -e "Today is \t${now}"
```

```
Today is          30_09_2025
```

```
mycomputer=${HOSTNAME}
```

```
myfiles=$(ls *awk*)
```

View contents

echo "\${myfiles}"

```
awkoutputfile.txt  
myawkscript.awk
```

echo \${myfiles}

```
awkoutputfile.txt  
myawkscript.awk
```

echo -e "### These are my files:\n\${myfiles}"

```
### These are my files:  
awkoutputfile.txt  
myawkscript.awk
```

ls -l *awk* > myfiles.list

cat myfiles.list

```
awkoutputfile.txt  
myawkscript.awk
```



We can import variables we have already made into an awk script

Introduce "external" variables using -v for each one

Each line of the input goes through the script one at a time

```
cat myfiles.list | awk -v Linuxbox=${mycomputer} -v now=${now} '{  
  print "As of " now " , " Linuxbox " has " $1 " stored on it."  
}'
```

```
As of 30_09_2025, bioinfmsc8.bio.ed.ac.uk has awkoutputfile.txt stored on it.  
As of 30_09_2025, bioinfmsc8.bio.ed.ac.uk has myawkscript.awk stored on it.
```

📌 The Internal Field Separator (IFS)

IFS is used for word/line splitting: it has default values of **space, tab, and newline**.

```
# Check the current values of our variable
```

```
echo ${now}
```

```
30_09_2025
```

```
# We can read the values, creating new variables from what we read in
```

```
read d1 d2 d3 <<< "${now}"
```

```
echo -e "${d1}"
```

```
30_09_2025
```

WHY??

```
# Specify our own delimiter!
```

```
IFS="_"
```

```
# Check the current values
```

```
echo ${now}
```

```
30_09_2025
```

```
# Read the values again, creating new variables
```

```
read d1 d2 d3 <<< "${now}"
```

```
echo -e "${d1}"
```

```
29
```

```
echo -e "${d2}"
```

```
09
```

```
echo -e "${d3}"
```

```
2025
```

```
# However...
```

```
unset d3
```

```
read d1 d2 <<< "${now}"
```

```
echo -e "${d1}"
```

```
29
```

```
echo -e "${d2}"
```

```
09_2025
```

```
echo -e "${d3}"
```

```
# Don't forget to reset/unset the separator:  
# this reverts it back to tab/space/newline!
```

```
unset IFS
```

bash `read` is a really useful/powerful utility, so do check out these links...

1. https://linuxhint.com/bash_read_command/
2. <https://linuxize.com/post/bash-read/>

⬆ Using escape characters

```
# PLEASE (please, please) don't do this for real (spaces, system variable names, etc...)
```

```
# This is more for the example....
```

```
mkdir "Where most of my executable files are is given in PATH"
```

```
cd Where most of my executable files are is given in PATH
```

```
-bash: cd: Where: No such file or directory
```

```
cd "Where most of my executable files are is given in PATH"
```

```
# Go back to where we were
```

```
cd -
```

```
cd Where\ most\ of\ my\ executable\ files\ are\ is\ given\ in\ PATH/
```

```
echo "Where most of my executable files are is given in \${PATH}"
```

```
Where most of my executable files are is given in ${PATH}
```

```
echo "Where most of my executable files are is given in ${PATH}"
```

```
Where most of my executable files are is given in /home/aivens2/edirect:/localdisk/home/ubuntu-  
software/SPAdes-3.15.5-Linux/bin:/opt/R/4.5.1/lib/R/bin:/localdisk/home/ubuntu-  
software/bowtie2-2.5.4:/localdisk/home/ubuntu-software/ncbi-blast-2.13.0+-  
src/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin;  
software/kaiju/bin:/localdisk/home/ubuntu-software/aliview:/localdisk/home/ubuntu-  
software/artemis:/localdisk/home/ubuntu-software/Augustus/bin:/localdisk/home/ubuntu-
```

software/BayesTraitsV4.0.0-Linux:/localdisk/home/ubuntu-
software/bbmap:/localdisk/home/ubuntu-software/bcftools-1.16/bin:/localdisk/home/ubuntu-
software/bedops_linux_x86_64-v2.4.41:/localdisk/home/ubuntu-
software/bedtools/bin:/localdisk/home/ubuntu-software/BRAKER-
2.1.6/scripts:/localdisk/home/ubuntu-software/bwa:/localdisk/home/ubuntu-
software/Anaconda3/bin:/localdisk/home/ubuntu-software/FastME/bin:/localdisk/home/ubuntu-
software/freebayes:/localdisk/home/ubuntu-software/gemma_0.98.5:/localdisk/home/ubuntu-
software/go/bin:/localdisk/home/ubuntu-software/hisat2-2.2.1:/localdisk/home/ubuntu-
software/hmmer-3.3.2:/localdisk/home/ubuntu-software/htslib-1.16:/localdisk/home/ubuntu-
software/iqtree-2.2.0-Linux/bin:/localdisk/home/ubuntu-
software/kallisto:/localdisk/home/ubuntu-software/kinfin-1.0.3:/localdisk/home/ubuntu-
software/MAFFT_7.505/bin:/localdisk/home/ubuntu-software/mcl-14-
137/bin:/localdisk/home/ubuntu-software/meme/bin:/localdisk/home/ubuntu-
software/meme/libexec/meme-5.5.7:/localdisk/home/ubuntu-
software/metaeuk/bin:/localdisk/home/ubuntu-software/Notung-
2.9.1.5:/localdisk/home/ubuntu-software/orthofinder_2.5.4:/localdisk/home/ubuntu-
software/php-8.2.19/bin:/localdisk/home/ubuntu-software/phylic-
3.697/exe:/localdisk/home/ubuntu-software/PhyML-4X:/localdisk/home/ubuntu-
software/picard:/localdisk/home/ubuntu-software/popoolation2_1201:/localdisk/home/ubuntu-
software/popoolation2_1201/Modules:/localdisk/home/ubuntu-software/primer3-
2.6.1/src:/localdisk/home/ubuntu-software/samclip:/localdisk/home/ubuntu-software/samtools-
1.16.1:/localdisk/home/ubuntu-software/selscan/bin/linux:/localdisk/home/ubuntu-
software/skewer-0.2.2:/localdisk/home/ubuntu-software/smalt:/localdisk/home/ubuntu-
software/sra-toolkit/bin:/localdisk/home/ubuntu-software/STAR-
2.7.3a/bin/Linux_x86_64_static:/localdisk/home/software/subread-2.0.8-Linux-
x86_64/bin:/localdisk/home/ubuntu-software/Tablet.1.21.02.08:/localdisk/home/ubuntu-
software/tophat-2.1.1.Linux_x86_64:/localdisk/home/ubuntu-software/trinityrnaseq-
v2.15.2:/localdisk/home/ubuntu-software/velvet_software:/localdisk/home/ubuntu-
software/vsearch-2.22.1-linux-x86_64-static/bin:/home/ubuntu-
software/wtdbg:/localdisk/home/ubuntu-software/HMMVE_1.2

📌 Simple mathematics: it's good not to be spaced out!

```
# Spaces are not what is wanted!
```

```
total_1= $(( 15 + 15 ))
```

```
bash: 30: command not found
```

```
# Spaces are not what is wanted (again)!
```

```
total_1 = $(( 15 + 15 ))
```

```
bash: total_1: command not found
```

```
# NO spaces either side of the = sign !
```

```
total_1=$(( 15 + 15 ))
```

```
total_2=$(( ${total_1} + 15 ))
```

```
echo ${total_1}
```

```
30
```

```
echo ${total_2}
```

```
45
```

Note that the "no spaces" rule also applies to when you set up variables. The variable values could have spaces, but the variable name certainly can not!

This is OK: `variableABC="A short sentence"`

This is OK: `variableABC="A_short_sentence"`

This is OK: `variableABC=123`

This is NOT OK: `variableABC= "A short sentence"`

This is NOT OK: `variableABC ="A short sentence"`

This is NOT OK: `variableABC=A short sentence`

[Give feedback](#)

[Ask for help](#)

This is NOT OK: `variable ABC="A short sentence"`

📌 Iteration with loops

```
ls -l *awk* > myfiles.list
```

```
cat myfiles.list
```

```
awkoutputfile.txt  
myawkscript.awk
```

Situation: we want to do the same thing with each file: count the number of lines, and output whatever we find.

```
# Short bash script to count the number of lines in one or more files
```

```
# Can you identify the input, process, output components?!
```

```
while read myfilename  
do  
echo -e "Processing ${myfilename}..."  
linesinfile=$(wc -l ${myfilename} | cut -d ' ' -f1)  
echo -e "\thas ${linesinfile} lines in it"  
done < myfiles.list
```

```
Processing awkoutputfile.txt...  
      has 2 lines in it  
Processing myawkscript.awk...  
      has 35 lines in it
```

```
# Short bash script to count the number of lines in files,
```

```
# multiple output "types"
```

```
while read myfilename  
do  
echo -e "Processing ${myfilename}..."  
wc_lines=$(wc -l ${myfilename})  
now=$(date)  
echo -e "${now} \t ${wc_lines}" >> howmanyline.txt  
linesinfile=$(wc -l ${myfilename} | cut -d ' ' -f1)  
echo -e "\thas ${linesinfile} lines in it"  
done < myfiles.list
```

```
# What is in the saved file?
```

```
cat howmanyline.txt
```

```
Tue 30 Sep 11:53:52 BST 2025    2 awkoutputfile.txt  
Tue 30 Sep 11:53:52 BST 2025   35 myawkscript.awk
```

Let's use our output from the last script!

Lots more variables this time: how many fields are there in each line?

awk '{print NF}' howmanylines.txt

8
8

What was the field separator that got used by awk in the above command?!



Let's give the variables some meaningful names

```
while read day numb month time timezone year linesfound filename
do
echo -e "${day}\t${timezone}\t${linesfound}"
done < howmanylines.txt
```

```
Tues    BST 2
Tues    BST 35
```



We can also use a "for" loop

```
for animal in cat dog llama
do
    echo ${animal}
done
```

```
cat
dog
llama
```

We can also use a "for" loop (single line, two variables, separate commands using ";")

for number in {1..4} ; do echo -e "\${number}\t\${animal}"; done

```
1      llama
2      llama
3      llama
4      llama
```

Why was it always a llama?

We can also use a "for" loop set up in a different way #1: steps of 5

```
for number in {0..20..5}
do
    echo -e "${number}\t${animal}"
done
```

```
0      llama
5      llama
10     llama
15     llama
20     llama
```

We can also use a "for" loop set up in a different way #2: steps of 5

```
for ((i = 0 ; i <= 50 ; i+=5)); do  
    echo -e "${i}\t${animal}"  
done
```

```
0      llama  
5      llama  
10     llama  
15     llama  
20     llama  
25     llama  
30     llama  
35     llama  
40     llama  
45     llama  
50     llama
```

We can also use multiple "for" loops inside each other (nested) # drawn to look nice

```
for number in {1..4} ;  
do  
    for animal in cat dog llama  
    do  
        echo -e "${number}\t${animal}"  
    done  
done
```

```
1      cat  
1      dog  
1      llama  
2      cat  
2      dog  
2      llama  
3      cat  
3      dog  
3      llama  
4      cat  
4      dog  
4      llama
```

Same set of commands, each separated by ";" but all on one line

```
for number in {1..4} ; do for animal in cat dog llama; do echo -e "${number}\t${animal}";done;done
```

We can also use multiple "for" loops inside each other (nested) # and a counter in the loop too

```
unset count  
for number in {1..4} ;  
do  
    for animal in cat dog llama  
    do  
        count=$((count+1)) # or ((count+=1)) or ((count++))  
        echo -e "${count}\t${number}\t${animal}"  
    done  
done
```

done		
done		
1	1	cat
2	1	dog
3	1	llama
4	2	cat
5	2	dog
6	2	llama
7	3	cat
8	3	dog
9	3	llama
10	4	cat
11	4	dog
12	4	llama

If you forget to do the `unset count` bit (which deletes our `${count}` variable value), this is what you would get the next time you run the script!

13	1	cat
14	1	dog
15	1	llama
16	2	cat
17	2	dog
18	2	llama
19	3	cat
20	3	dog
21	3	llama
22	4	cat
23	4	dog
24	4	llama

A good summary of things we can do with bash loops is available at <https://linuxize.com/post/bash-for-loop/>

↑ Conditionals: if...

We frequently have a situation that is usually something like:

```
if [ condition ] ;  
then  
do something  
else  
do somethingelse  
fi
```

How can we do it in bash?

Here are the top few lines of the BLAST output file `blastoutput2.out` we used earlier.

head blastoutput2.out

```
# BLASTX 2.5.0+  
# Query: NC_015119.1:1426-2962 Boreus elegans mitochondrion, complete genome  
# Database: nem  
# Fields: query acc., subject acc., % identity, alignment length, mismatches, gap opens, q. start, q. end, s. start, s. end, evalue, bit score  
# 405 hits found  
NC_015119.1:1426-2962 gi|225622197|ref|YP_002725710.1| 61.712 444 169 1 4 1335 12 454 7.36e-147  
NC_015119.1:1426-2962 gi|288903410|ref|YP_003434131.1| 61.991 442 167 1 10 1335 14 454 3.08e-146  
NC_015119.1:1426-2962 gi|326536058|ref|YP_004300493.1| 62.443 442 165 1 10 1335 14 454 3.29e-146  
NC_015119.1:1426-2962 gi|188011119|ref|YP_001905892.1| 61.312 442 170 1 10 1335 14 454 8.19e-146  
NC_015119.1:1426-2962 gi|225622184|ref|YP_002725698.1| 61.486 444 170 1 4 1335 12 454 1.06e-145
```

Process some BLAST output data

Here, we are going to use the `test` option, described in more detail on [here](#).

We are also going to do some comparisons in bash, and we can do these comparisons on bits of text ("strings"), or numbers.

The syntax for doing comparisons is described in more detail [here](#).

We will write a short script called `good_or_bad_BLAST.sh` in the current folder. The script will have the following contents:

```
#!/bin/bash
```

```
# Reset IFS back to usual defaults (just in case it was still "_" from before)
```

```
unset IFS
```

```
# input, process, output
```

```
# Process using a pipe
```

```
cut -f1,12 blastoutput2.out | grep -v "#" |
while read bn1 bn2
do
  # Evaluation step
  if test ${bn2} -ge 300
  then
    echo "HSP bit score ${bn2} is truly excellent"
    echo -e "${bn1}\t${bn2}" >> good.blastn
  else
    echo "HSP bit score ${bn2} is not excellent"
    echo -e "${bn1}\t${bn2}" >> bad.blastn
  fi
done
```

Now, we need to make our script executable and then run it.

```
chmod 755 good_or_bad_BLAST.sh
```

```
./good_or_bad_BLAST.sh
```

```
HSP bit score 432 is truly excellent
HSP bit score 430 is truly excellent
HSP bit score 430 is truly excellent
HSP bit score 429 is truly excellent
HSP bit score 429 is truly excellent
HSP bit score 429 is truly excellent
HSP bit score 429 is truly excellent
....
HSP bit score 112 is not excellent
HSP bit score 107 is not excellent
./good_or_bad_BLAST.sh: line 12: test: 90.5: integer expression expected
HSP bit score 90.5 is not excellent
....
```

We are getting errors, because bash doesn't like (can't do) floating point comparisons to integers....so let's fix that by just keeping the value BEFORE the decimal point, i.e. the integer bit (it will be accurate enough for us here).

All we have to do is modify the value of `${bn2}` as we read in each line of input. To do this we can take advantage of some `printf` functionality, essentially making a floating point number into an integer. Here is the new version of

```
good_or_bad_BLAST.sh
```

```
#!/bin/bash
```

```
# Reset IFS back to usual defaults (just in case it was still "_" from before)
```

unset IFS

Remove our previous outputs, then run it again

```
rm *.blastn
```

```
# input, process, output
```

```
# Process using a pipe
```

```
cut -f1,12 blastoutput2.out | grep -v "#" |
while read bn1 bn2
do
  bn2=$(printf %0.f ${bn2})
  # Evaluation step
  if test ${bn2} -ge 300
  then
    echo "HSP bit score ${bn2} is truly excellent"
    echo -e "${bn1}\t${bn2}" >> good.blastn
  else
    echo "HSP bit score ${bn2} is not excellent"
    echo -e "${bn1}\t${bn2}" >> bad.blastn
  fi
done
```

```
...
HSP bit score 62 is not excellent
HSP bit score 57 is not excellent
HSP bit score 49 is not excellent
HSP bit score 40 is not excellent
HSP bit score 40 is not excellent
HSP bit score 35 is not excellent
HSP bit score 29 is not excellent
HSP bit score 30 is not excellent
```

ls -al *.blastn

```
-rw-r--r-- 1 aivens2 g_aivens2 8388 Sep 29 12:17 bad.blastn
-rw-r--r-- 1 aivens2 g_aivens2 2080 Sep 29 12:17 good.blastn
```

head good.blastn

```
NC_015119.1:1426-2962    432
NC_015119.1:1426-2962    430
NC_015119.1:1426-2962    430
NC_015119.1:1426-2962    429
NC_015119.1:1426-2962    429
NC_015119.1:1426-2962    429
NC_015119.1:1426-2962    429
NC_015119.1:1426-2962    428
NC_015119.1:1426-2962    428
```

There's more details about using `if` statements in bash [here](#) and we'll be using them quite a lot over the next few lecture/workshops, so it would be a good idea to have at least a look at it: knowledge is power!

📌 OK, nearly done, it's tidy up and git commit time...!

ls -l

```
awkoutputfile.txt
bad.blastn
blastoutput2.out
good.blastn
good_or_bad_BLAST.sh
howmanylines.txt
myawkscript.awk
myfiles.list
myinputfile.tsv
```

What is our git status?

git st

```
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    ../Lecture04/CodeFiles.tar.gz

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ../Lecture04/Als_ICA/
    ../Lecture04/all_the_things_I_did
    ../Lecture04/outs/
    awkoutputfile.txt
    bad.blastn
    blastoutput2.out
    good.blastn
    good_or_bad_BLAST.sh
    howmanylines.txt
    myfiles.list
    myinputfile.tsv

no changes added to commit (use "git add" and/or "git commit -a")
```

We decide to add three of them to our repository

```
git add howmanylines.txt good_or_bad_BLAST.sh good.blastn
```

Now, commit with an informative message

```
git commit -m "First attempt at parsing BLAST data"
```

```
[master c9b5b18] First attempt at parsing BLAST data
 3 files changed, 104 insertions(+)
 create mode 100644 Lecture05/good.blastn
```

```
create mode 100755 Lecture05/good_or_bad_BLAST.sh
create mode 100644 Lecture05/howmanylines.txt
```

Check the status

git st

On branch master

Changes not staged for commit:

```
(use "git add/rm <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
deleted:    ../Lecture04/CodeFiles.tar.gz
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
../Lecture04/Als_ICA/
../Lecture04/all_the_things_I_did
../Lecture04/outs/
awkoutputfile.txt
bad.blastn
blastoutput2.out
myfiles.list
myinputfile.tsv
```

no changes added to commit (use "git add" and/or "git commit -a")

Prepare for cleaning up: dry run first....

git clean -n

```
Would remove awkoutputfile.txt
Would remove bad.blastn
Would remove blastoutput2.out
Would remove myfiles.list
Would remove myinputfile.tsv
```

Actually, I'd like to keep one of those...

git commit myinputfile.tsv -m "Decided to keep myinputfile.tsv"

error: pathspec 'myinputfile.tsv' did not match any file(s) known to git.

Doh! Forgot to stage the file first....

git add myinputfile.tsv

git commit -m "Decided to keep myinputfile.tsv"

```
[master 08d379b] Decided to keep myinputfile.tsv
1 file changed, 3 insertions(+)
create mode 100644 Lecture05/myinputfile.tsv
```

git clean -n

```
Would remove awkoutputfile.txt
Would remove bad.blastn
Would remove blastoutput2.out
Would remove myfiles.list
```

git add blastoutput2.out

git commit -m "Decided to keep blastoutput2.out"

```
[master 137c3db] Decided to keep blastoutput2.out
1 file changed, 411 insertions(+)
create mode 100644 Lecture05/blastoutput2.out
```

git clean -f

```
Removing awkoutputfile.txt
Removing bad.blastn
Removing myfiles.list
```

ls -l

```
blastoutput2.out
good.blastn
good_or_bad_BLAST.sh
howmanyline.txt
myawkscript.awk
myinputfile.tsv
```

git log

```
137c3db (HEAD -> master) Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeaa I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Can I get the missing files back?

undelete awkoutputfile.txt

-bash: undelete: command not found

What happens if I then make other files somewhere else in the "repo area"

touch \${HOME}/Exercises/BN.sh

touch \${HOME}/Exercises/BN.out

Status check

git st

```
On branch master
Changes not staged for commit:
```

```
(use "git add/rm <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
deleted:    ../Lecture04/CodeFiles.tar.gz
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
../BN.out
../BN.sh
../Lecture04/Als_ICA/
../Lecture04/all_the_things_I_did
../Lecture04/outs/
```

no changes added to commit (use "git add" and/or "git commit -a")

What should we do?

Would cleaning help? No.

git clean -n

What is there in the Exercises folder?

Is \${HOME}/Exercises

BN.out BN.sh Lecture04 Lecture05

Let's put copies of the "BN files" in our current working directory

cp \${HOME}/Exercises/BN.* .

Status check

git st

On branch master

Changes not staged for commit:

```
(use "git add/rm <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
deleted:    ../Lecture04/CodeFiles.tar.gz
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
../BN.out
../BN.sh
../Lecture04/Als_ICA/
../Lecture04/all_the_things_I_did
../Lecture04/outs/
BN.out
BN.sh
```

no changes added to commit (use "git add" and/or "git commit -a")

Would cleaning help?

git clean -n

Would remove BN.out
Would remove BN.sh

What happens if we do it!?

git clean -f

```
Removing BN.out
Removing BN.sh
```

Status check

git st

```
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    ../Lecture04/CodeFiles.tar.gz

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ../BN.out
    ../BN.sh
    ../Lecture04/Als_ICA/
    ../Lecture04/all_the_things_I_did
    ../Lecture04/outs/

no changes added to commit (use "git add" and/or "git commit -a")
```



Draw a text picture/graph of your repository structure

git log --graph --oneline --decorate --all

```
* 137c3db (HEAD -> master) Decided to keep blastoutput2.out
* 08d379b Decided to keep myinputfile.tsv
* c9b5b18 First attempt at parsing BLAST data
* bb8f825c My first awk script, Lecture05
* 68f27fb There were conflicts with someotherfile.sh, kept all changes
|\
| * 83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
* | 0b5425b (tag: v2.0) Updated contents of tar, coolscript
* | 5d8aeea I am not sure cool script is actually any good
* | bbd36e7 Added my_cool_script
* | ea60357 Third version of the someotherfile.sh shell script
* | b6ae4a7 Updated version of tar
|/
* ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
* a5add7c Added the someotherfile.sh shell script
* 101b700 Additional motif files, and updated tar file
* d21cb3d Adding intial tar file
* 59d1756 CodeFiles all done and tar zipped
* 6dd85a8 First file added
```



Set up an alias for it (adog)

git config --global alias.adog "log --all --decorate --oneline --graph"

git adog

```
* 137c3db (HEAD -> master) Decided to keep blastoutput2.out
* 08d379b Decided to keep myinputfile.tsv
* c9b5b18 First attempt at parsing BLAST data
* bb8f825c My first awk script, Lecture05
* 68f27fb There were conflicts with someotherfile.sh, kept all changes
|\
| * 83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
* | 0b5425b (tag: v2.0) Updated contents of tar, coolscript
```



```
* | 5d8aeaa I am not sure cool script is actually any good
* | bbd36e7 Added my_cool_script
* | ea60357 Third version of the someotherfile.sh shell script
* | b6ae4a7 Updated version of tar
|/
* ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
* a5add7c Added the someotherfile.sh shell script
* 101b700 Additional motif files, and updated tar file
* d21cb3d Adding intial tar file
* 59d1756 CodeFiles all done and tar zipped
* 6dd85a8 First file added
```



A prettier version with colours and more details (alias adog2)

```
git config --global alias.adog2 "log --graph --abbrev-commit --decorate
--format=format:'%C(yellow)%h%C(reset)%C(auto)%d%C(reset)
%C(normal)%s%C(reset) %C(dim white)%an%C(reset)
%C(dim blue)(%ar)%C      (reset)'" --all"
```

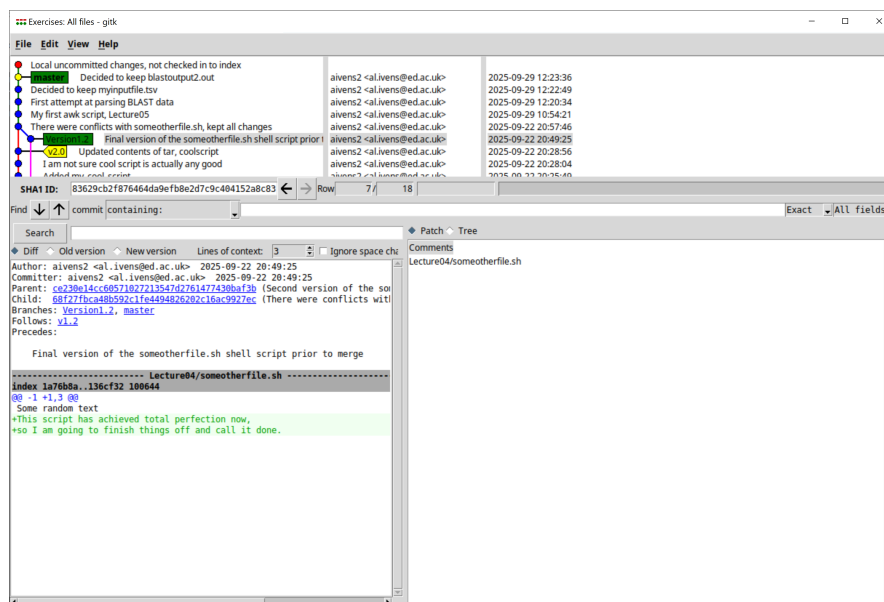
git adog2

```
* 137c3db (HEAD -> master)
|   Decided to keep blastoutput2.out aivens2
|   (7 minutes ago)%C      (reset)
* 08d379b
|   Decided to keep myinputfile.tsv aivens2
|   (8 minutes ago)%C      (reset)
* c9b5b18
|   First attempt at parsing BLAST data aivens2
|   (10 minutes ago)%C      (reset)
* bbf825c
|   My first awk script, Lecture05 aivens2
|   (2 hours ago)%C      (reset)
* 68f27fb
|\   There were conflicts with someotherfile.sh, kept all changes aivens2
|   (7 days ago)%C      (reset)
| * 83629cb (Version1.2)
|   Final version of the someotherfile.sh shell script prior to merge aivens2
|   (7 days ago)%C      (reset)
* | 0b5425b (tag: v2.0)
|   Updated contents of tar, coolscript aivens2
|   (7 days ago)%C      (reset)
* | 5d8aeaa
|   I am not sure cool script is actually any good aivens2
|   (7 days ago)%C      (reset)
* | bbd36e7
|   Added my_cool_script aivens2
|   (7 days ago)%C      (reset)
* | ea60357
|   Third version of the someotherfile.sh shell script aivens2
|   (7 days ago)%C      (reset)
* | b6ae4a7
|/   Updated version of tar aivens2
|   (7 days ago)%C      (reset)
* ce230e1 (tag: v1.2)
|   Second version of the someotherfile.sh shell script aivens2
|   (7 days ago)%C      (reset)
* a5add7c
|   Added the someotherfile.sh shell script aivens2
|   (7 days ago)%C      (reset)
* 101b700
|   Additional motif files, and updated tar file aivens2
|   (7 days ago)%C      (reset)
* d21cb3d
|   Adding intial tar file aivens2
|   (7 days ago)%C      (reset)
```

```
* 59d1756
| CodeFiles all done and tar zipped aivens2
| (7 days ago) %C      (reset)
* 6dd85a8
| First file added aivens2
| (7 days ago) %C      (reset)
```

The plain text picture can be very useful once you start branching etc, to help you "see where you are". A good link for this is where I took the above example from: <https://stackoverflow.com/questions/1057564/pretty-git-branch-graphs>.

Ideas/code for drawing nice pictures/exploring the repository can also be found here <https://stackoverflow.com/questions/1838873/visualizing-branch-topology-in-git/34467298#34467298> using `gitk` or `gitg`, for example. These are installed on our servers. All you have to do is to change directory to where your git repository is on the server, then type `gitk &`, and after a few seconds, a graphical representation of your repo will appear.



Exercises for this lecture

I'd like you to use the [Linux exercise materials](#) again, and do them with using git and GitHub in mind, i.e. think about how you might want to set things up, then try the exercises, committing your scripts to the local repository that you have made.

📌 Links you will/might need

- [Linux 2 exercises](#)
- [git/GitHub lecture](#)
- [Server log in](#)

Within a new directory, try doing the following things in the list below. You will find that these have to be done in more or less this order, otherwise you might not get what you think you are going to get!

1. if you havent already done so, initialise a git repository in your Exercises folder, make a .gitignore if you wish
2. set the various local and global parameters that you want; I would recommend that for the exercises, you use your personal GitHub account, not the Bxxxxxx-2025 one you might use for the optional ICA!
3. create a directory for today's exercises and change directory to it (I'd suggest calling it `${HOME}/Exercises/Lecture05`)
4. make, add, and commit various files that you generate while doing the Linux exercises
5. if you are feeling brave, try making a branch and set up a known conflict so that you can resolve it and merge it back into the master
6. create a new repository called BPSM_Lecture05 in your **GitHub** account area using the Linux command line. There are several ways you can do it:

1.

```
curl -u ${mypersonalGitHubusername} https://api.github.com/user/repos -d '{"name":"BPSM_Lecture05"}'
```

for this example, you will need to have set up your user name as a variable called *mypersonalGitHubusername* and you will need to paste/type in your personal account git token when asked

2.

```
curl -u ${mypersonalGitHubusername}:${mypersonalgittoken} https://api.github.com/user/repos -d '{"name":"BPSM_Lecture05"}'
```

for this example, in addition to the variable *mypersonalGitHubusername* you will need to have assigned your personal account git token to a variable called *mypersonalgittoken*

3.

```
curl -u ${mypersonalgithub_credentials} https://api.github.com/user/repos -d '{"name":"BPSM_Lecture05"}'
```

for this example, you will need to set up your user name and personal account git token as a single variable called *mypersonalgithub_credentials*

```
mypersonalgithub_credentials="${mypersonalGitHubusername}:${myperso
```

7. push your **local** repository to your **remote personal** GitHub space : it DOESN'T need to be encrypted, that was just to illustrate for the purposes of the optional ICA where we **WILL** want it to be encrypted!

Step 1: tell git which GitHub you want to call "origin"

```
git remote add origin
```

```
https://github.com/mypersonalGitHubusername/BPSM_Lecture05.git
```

Step 2: "push" your local repository (master) to GitHub (origin)

```
git push -u origin master
```

Username for 'https://github.com':

SuperCoder

Password for 'https://SuperCoder@github.com':

.....

```
Enumerating objects: 76, done.
Counting objects: 100% (76/76), done.
Delta compression using up to 256 threads
Compressing objects: 100% (69/69), done.
Writing objects: 100% (76/76), 16.95 KiB | 578.00 KiB/s, done.
Total 76 (delta 26), reused 0 (delta 0)
remote: Resolving deltas: 100% (26/26), done.
To https://github.com/SuperCoder/BPSM_Lecture05.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

Check it's all OK: it doesn't ask for your user name again



git push -u origin master

```
From https://github.com/SuperCoder/BPSM_Lecture05
* branch      master      -> FETCH_HEAD
Already up-to-date.
```

8. tell someone else about it so they can try to clone it: all they need to try is the following single-line command, where your repository is cloned from GitHub into a folder called `canIclonetheirs` in their homespace:

git clone

```
https://github.com/someone\_elses\_personalGitHubusername/BPSM\_Lecture05.git
${HOME}/canIclonetheirs
```

NOW, let's do some real coding: try the following exercises, which uses the same data set as we used in the Linux 2 lecture. This way of doing things is worthy of consideration as a general approach for all the things we do, including the "My First Pipeline" exercise and the optional Python ICA...

Exercise

🏠 Process a file line by line using bash shell tools

Process the `example_people_data.tsv` to put people into different groups (you chose which!) using a bash script.

Copy the file called

`example_people_data.tsv`

which is in the

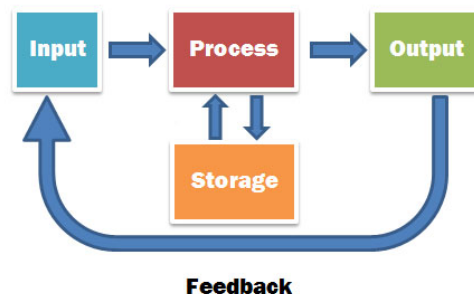
`/localdisk/data/BPSM/Lecture03`

directory, into your exercises directory for this lecture.

The file is a tab-delimited plain text file, and the column headings are

```
name    email    city    birthday_day    birthday_month    birthday_year    country
```

As you already know, this can be done with awk/grep (or R or Java or C or Perl or....) but you are **not allowed** to use awk or grep etc this time: bash only!



Suggested order of doing things

1. Think about the problem, what first group you might want to choose
2. Break the problem into bits if you can
3. Draw a rough outline of the process you have in mind
4. Repeat 1-3 as necessary
5. Start coding, add and commit to git as appropriate (frequently)
6. Test coding with data that give known answers
7. Repeat 4-6 as necessary

I will give possible answers for the following explorations of the data:

- Go through the file showing an index counter and the country
- Go through the file showing an index counter, name, city and the country, but without the header and blanks lines this time
- Put people into separate files: a different file for each country
- Work out how many people were born in October, and where are they from, output the list of people
- Work out how many people were born in October, and where are they from, output as multiple lists
- Using an array to process the Mozambique data (have not told you how to do this, it's just something for those who want more...)

Of course, we are now getting better at using a plain text editor in the Linux command line environment so we can write our own scripts in a 'nix environment... anywhere... we know the `vi` and `nano` software is already installed on our servers...!

- [vi \(vi cheatsheet\)](#)
- [nano \(nano cheatsheet\)](#)

Bear in mind that you do now know how to edit and save files, and then make them executable, so there should be less and less need to copy-paste things (if at all...)!

... don't forget to add and commit your scripts to git regularly with useful, informative messages

... and push to GitHub...I will check that you are doing this, at some point, so be warned... 

(Possible) Solutions

Get the file of random people's details (faked):

Copy the file called

```
example_people_data.tsv
```

which is in the

```
/localdisk/data/BPSM/Lecture03
```

directory, into your exercises directory for this lecture.

The file is a tab-delimited plain text file, and the column headings are

```
name    email    city    birthday_day    birthday_month    birthday_year    country
```

Process the `example_people_data.tsv` to put people into different groups (you chose which!) using a bash script.

I will give possible answers for these explorations of the data:

- Go through the file showing an index counter and the country
- Go through the file showing an index counter, name, city and the country, but without the header and blanks lines this time
- Put people into separate files, a different file for each country
- Work out how many people were born in October, and where are they from, output the list of people
- Work out how many people were born in October, and where are they from, output as multiple lists
- Using an array to process the Mozambique data (have not told you how to do this...)

... don't forget to add and commit your scripts to git with useful, informative messages...!

```
# Initial set up
```

```
cd
mkdir -p Exercises/Lecture05
cd ~/Exercises/Lecture05
cp /localdisk/data/BPSM/Lecture03/example_people_data.tsv .
```

Check the file is still the one we wanted

head -5 example_people_data.tsv

name	email	city	birthday_day	birthday_month	birthday_year	country
Axel	nulla.at@auguescelerisquemollis.net	Latinne	5	12	1978	Tokelau
Karleigh	congue@auctornuncnulla.co.uk	Aurora	8	3	1958	Kenya
Celeste	lacus@diam.com	Stirling	17	10	1984	Grenada
Josephine	dui@faucibuslectus.net	Le Petit-Quevilly	2	8	1985	Sudan

It was! Let's put it in our repository.

Stage and commit

git add example_people_data.tsv

git commit -m "Added the example_people_data.tsv file from Lecture03"

```
[master 19cdd2d] Added the example_people_data.tsv file from Lecture03
1 file changed, 106 insertions(+)
create mode 100755 Lecture05/example_people_data.tsv
```

Current logfile

git log

```
19cdd2d (HEAD -> master) Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeaa I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Which directory are we in?

pwd

```
/home/s000000/Exercises/Lecture05
```

Go through the file showing an index counter and the country

```
count=0
while read name email city birthday_day birthday_month birthday_year country
do
count=$((count+1))
```

```
echo -e "${count}\t${country}"
done < example_people_data.tsv | head -10

1      country
2      Tokelau
3      Kenya
4      Grenada
5      1985      Sudan
6      United Arab Emirates
7      Saint Helena, Ascension and Tristan da Cunha
8      Peru
9      United States
10     Equatorial Guinea
```

and now the end few lines of the output

```
count=0;
while read name email city birthday_day birthday_month birthday_year country
do
    count=$((count+1))
    echo -e "${count}\t${country}"
done < example_people_data.tsv | tail -10

97     Maldives
98     Armenia
99     Mauritius
100    Kazakhstan
101    Bolivia
102
103
104
105
106
```

Did we forget to reset IFS? We only want a tab character as the separator

unset IFS

IFS=\$'\t'

Try again...!

```
count=0
while read name email city birthday_day birthday_month birthday_year country
do
    count=$((count+1))
    echo -e "${count}\t${country}"
done < example_people_data.tsv | head -10

1      country
2      Tokelau
3      Kenya
4      Grenada
5      Sudan
6      United Arab Emirates
7      Saint Helena, Ascension and Tristan da Cunha
8      Peru
9      United States
10     Equatorial Guinea
```

Go through the file showing an index counter, name, city and
the country, but without the header and blank lines this time


```
count=0
IFS=$'\t'
while read name email city birthday_day birthday_month birthday_year country
do

# test -z enables a test for an empty variable: true or false!
if test -z ${name}
then
echo -e "X\tBlank line found"

# variable had a value, so was it the word "country" or not
else
if test ${country} == "country"
then
echo -e "X\tHeader line found"
else
count=$((count+1))
echo -e "${count}\t${country}"
fi # a real country
fi # a blank line
done < example_people_data.tsv

X      Header line found
1      Tokelau
2      Kenya
3      Grenada
4      Sudan
5      United Arab Emirates
... other lines
99     Kazakhstan
100    Bolivia
X      Blank line found
X      Blank line found
X      Blank line found
X      Blank line found
X      Blank line found
```

So.... now we know for sure that there were 100 people in the list...! 😊

```
# Lets put people into files for each country, combining evaluations

count=0; IFS=$'\t';
while read name email city birthday_day birthday_month birthday_year country
do
if test -z ${name} || test ${country} == "country"
then
count=count; # a meaningless action, it does nothing useful
else
echo "outputfile1 will be ${country}.details";

# Need to remove/substitute any spaces in the variable value
# because we want to use the variable as part of a filename
echo "outputfile2 will be ${country// /}.details"
fi
done < example_people_data.tsv | head -10

outputfile1 will be Tokelau.details
outputfile2 will be Tokelau.details
outputfile1 will be Kenya.details
outputfile2 will be Kenya.details
outputfile1 will be Grenada.details
outputfile2 will be Grenada.details
outputfile1 will be Sudan.details
outputfile2 will be Sudan.details
```

```
outputfile1 will be United Arab Emirates.details
outputfile2 will be UnitedArabEmirates.details
```

OK, that worked, let's do it for real

count=0; IFS=\$'\t';

remove any existing output files

```
rm -f *.details
while read name email city birthday_day birthday_month birthday_year country
do
if test -z ${name} || test ${country} == "country"
then
echo "Ignoring"
else
count=$((count+1));
# To replace all occurrences of some character use ${parameter//pattern/string}

echo -e "${count}\t${name}\t${city}\t${country}" >> ${country// /}.details
fi
done < example_people_data.tsv
```

```
Ignoring
Ignoring
Ignoring
Ignoring
Ignoring
Ignoring
```

Did we get what we were expecting?

cat Mozambique.details

13	Chava	Lusevera	Mozambique
71	Chloe	Pergola	Mozambique
74	Zorita	Massarosa	Mozambique
83	Cally	Rocky View	Mozambique

Country and age grouping this time, also using continue

I arbitrarily decided that anyone born before 1980 was "older"

```
count=0; IFS=$'\t';
rm -f *.details
while read name email city birthday_day birthday_month birthday_year country
do
if test -z ${name} || test ${country} == "country"
then
continue
else
count=$((count+1));
```

Set a default output file name for "younger" people

```
outputfile=${country// /}.younger.details
if test ${birthday_year} -le 1980
then
outputfile=${country// /}.older.details
fi # birthday before 1980, so "older" person
echo -e "${count}\t${name}\t${birthday_year}\t${country}" >> ${outputfile}
fi
done < example_people_data.tsv
```

Did we get what we were expecting?

Is *.details

Afghanistan.younger.details	Ghana.older.details	Norway.younger.details
Albania.older.details	Greenland.younger.details	Pakistan.younger.details
Andorra.older.details	Grenada.younger.details	Panama.younger.details
AntiguaandBarbuda.older.details	Guadeloupe.older.details	Paraguay.older.details
Armenia.older.details	Guam.older.details	Peru.younger.details
Armenia.younger.details	Guinea-Bissau.older.details	PitcairnIslands.older.details
Australia.younger.details	Guinea.younger.details	PuertoRico.older.details
Bangladesh.older.details	Guyana.older.details	Romania.older.details
Bhutan.older.details	HeardIslandandMcDonaldIslands.younger.details	SaintHelena,AscensionandTristandaCunha.older.details
Bolivia.older.details	HongKong.younger.details	SaintKittsandNevis.younger.details
Botswana.younger.details	IsleofMan.older.details	SaintLucia.older.details
BouvetIsland.older.details	Japan.older.details	SaintPierreandMiquelon.older.details
Brazil.younger.details	Kazakhstan.younger.details	SaoTomeandPrincipe.older.details
Brunei.younger.details	Kenya.older.details	SaoTomeandPrincipe.younger.details
Congo(Brazzaville).younger.details	Kyrgyzstan.older.details	SouthGeorgiaandTheSouthSandwichIslands.younger.details
CookIslands.older.details	Libya.younger.details	Sudan.younger.details
Cuba.younger.details	Macedonia.older.details	Tokelau.older.details
Curacao.older.details	Malaysia.older.details	TrinidadandTobago.older.details
Curacao.younger.details	Maldives.older.details	TurksandCaicosIslands.younger.details
Cyprus.older.details	Martinique.younger.details	UnitedArabEmirates.older.details
Egypt.older.details	Mauritania.younger.details	UnitedKingdom.older.details
Egypt.younger.details	Mauritius.older.details	UnitedStates.older.details
ElSalvador.older.details	Micronesia.younger.details	VietNam.older.details
EquatorialGuinea.younger.details	Mongolia.younger.details	VirginIslands,British.older.details
Eritrea.older.details	Mozambique.older.details	VirginIslands,UnitedStates.older.details
FalklandIslands.older.details	Mozambique.younger.details	WallisandFutuna.younger.details
Fiji.older.details	Myanmar.older.details	WesternSahara.older.details
Finland.older.details	NewCaledonia.older.details	Zambia.younger.details
FrenchPolynesia.younger.details	NewCaledonia.younger.details	Zimbabwe.younger.details
Georgia.older.details	Nicaragua.older.details	

wc -l Mozambique.*

```
3 Mozambique.older.details
1 Mozambique.younger.details
4 total
```

cat Mozambique.older.details

13	Chava	1971	Mozambique
71	Chloe	1978	Mozambique
74	Zorita	1979	Mozambique

cat Mozambique.younger.details

83	Cally	1992	Mozambique
----	-------	------	------------

How many were born in October, and where are they from

```
count=0; IFS=$'\t';
```

Set additional variables this time (we might want to change these later)

```
month=10
outputfile="Month.$month.details"
```

```
rm -f *.details
while read name email city birthday_day birthday_month birthday_year country
do
if test -z ${name} || test ${country} == "country"
then
echo "Ignoring"
else
count=$((count+1));
```

Comparing numbers, as month is actually a number

```
if test ${birthday_month} -eq $month
then
    echo -e "${count}\t${name}\t${birthday_month}\t${country}" >> ${outputfile}
fi # birthday month
fi
done < example_people_data.tsv
```

Did we get what we were expecting?

wc -l Month.10.details

11 Month.10.details

head -5 Month.10.details

3	Celeste	10	Grenada
19	Vivien	10	Sao Tome and Principe
43	Bruce	10	Afghanistan
49	Berk	10	Congo (Brazzaville)
53	Leilani	10	Japan

sort -k4,4 Month.10.details | head -4

43	Bruce	10	Afghanistan
95	Grace	10	Antigua and Barbuda
82	Fulton	10	Brunei
49	Berk	10	Congo (Brazzaville)

cut -f4 Month.10.details | sort | uniq -c | sort -k1,1nr | head -5

1	Afghanistan
1	Antigua and Barbuda
1	Brunei
1	Congo (Brazzaville)
1	Grenada

We could also use an array to store our results as the script processes the data. There are many online resources, for example: [this one](#).

Process the Mozambique data

Set lots of variables

```
count=0; fnr=0; IFS=$'\t';
wantedcountry="Mozambique"
inputfile="example_people_data.tsv"
inputfilelength=$(wc -l ${inputfile} | cut -d ' ' -f1)
outputfile="Country.${wantedcountry}.details"
```

Initial actions

```
rm -f *.details
unset my_array
declare -A my_array
```

Main actions

```
while read name email city birthday_day birthday_month birthday_year country
do
    fnr=$((fnr + 1))
```

```
# echo "Line number: ${fnr}"
if test -z ${name} || test ${country} == "country"
then
echo "" > /dev/null
else
if test ${country} == ${wantedcountry}
then
count=$((count+1));
my_array[${count}]="${fnr}\t${name}\t${country}"
# echo -e "${my_array[${count}]}"
fi
fi
# End of the file
if test ${fnr} -eq ${inputfilelength}
then
echo -e "\n### Here are the end of file results for ${wantedcountry}:" > ${outputfile}
for i in "${my_array[@]}"; do echo -e "$i" >> ${outputfile}; done
fi
done < ${inputfile}
```

Did we get what we were expecting?

cat \${outputfile}

```
### Here are the end of file results for Mozambique:
84      Cally      Mozambique
75      Zorita     Mozambique
72      Chloe      Mozambique
14      Chava      Mozambique
```

Same command...

cat Country.\${wantedcountry}.details

```
### Here are the end of file results for Mozambique:
84      Cally      Mozambique
75      Zorita     Mozambique
72      Chloe      Mozambique
14      Chava      Mozambique
```

Same command...

cat Country.Mozambique.details

```
### Here are the end of file results for Mozambique:
84      Cally      Mozambique
75      Zorita     Mozambique
72      Chloe      Mozambique
14      Chava      Mozambique
```

Maybe we could ask the user what country to process?!

read -p "What country shall we process the data for?" wantedcountry

What country shall we process the data for?

China

```
echo ${wantedcountry}
```

```
China
```

```
echo -n "What country shall we process the data for? " ; read wantedcountry
```

```
What country shall we process the data for?
```

```
UK
```

```
echo ${wantedcountry}
```

```
UK
```

... and so on ... !

Finally...

git st

On branch master

Changes not staged for commit:

(use "git add/rm <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

deleted: ../Lecture04/CodeFiles.tar.gz

Untracked files:

(use "git add <file>..." to include in what will be committed)

../BN.out

../BN.sh

../Lecture04/Als_ICA/

../Lecture04/all_the_things_I_did

../Lecture04/outs/

Country.Mozambique.details

no changes added to commit (use "git add" and/or "git commit -a")

... don't forget to add and commit important files (for example, your scripts?!) to git ... and push to GitHub... the instructions are all given above...

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!

